

Programming Project 1 - Sorting Algorithms

Name: Connor

Course: CS 2430

Semester: Summer 2026

There Are 4 algorithms and they mostly all do the same things the real problem is how efficient they are when you have mass amounts of data.

Array Generator

create an array of random numbers

Merge Sort

Splits the array until it only has one piece then reassembles in sorted order.

Quick Sort

Gets a high low and pivot compares then moves in once the low is greater than the high it can sort that element then passes a smaller array down.

Heap Sort

Builds a max heap putting the largest at the root then pulls it off swaps it to the back and fixes the structure. Does this until sorted.

Shaker Sort

Bubble sort but goes both directions. Forward pass moves the biggest to the right backward pass moves the smallest to the left.

Comparison Counting

Every time two values are directly compared the counter goes up.

All four algorithms were checked by printing the before and after arrays on several separate runs. Every one of them produced correct ascending output every time. Single element arrays were also tested and none of the algorithms broke or did any unnecessary work on them.

10 trials per size arrays randomly generated each run.

10 Runs size = 4

PROCESS MIN MAX AVG

Merge: 4 5 4.80

Quick: 4 6 4.90

Heap: 2 6 3.50

Shaker: 6 6 6.00

10 Runs size = 6

PROCESS MIN MAX AVG

Merge: 8 11 9.50
Quick: 8 14 10.40
Heap: 5 9 7.30
Shaker: 15 15 15.00
10 Runs size = 8
PROCESS MIN MAX AVG
Merge: 14 17 15.80
Quick: 14 27 18.10
Heap: 14 20 16.20
Shaker: 28 28 28.00

Easiest to implement?

Shaker sort. Already knew bubble sort so the two direction version was pretty easy.

Hardest to implement?

Merge sort. Least familiar with this one had a harder time and had to debug

How were comparisons counted and why does it matter?

the counter goes up every time there is a comparison

Which was most efficient?

Heap and merge sort because they are $(n \log n)$ and shaker is $O(n^2)$

Why don't small sizes perfectly match Big-O?

because when you have such small numbers there is not much impact no matter what you use

What would stronger evidence look like?

when you use bigger arrays or do it a bunch of times

What implementation choices affect performance?

most sorting algorithm depend on how the array is when it is sent for quick sort if it is already sorted it turns into $O(n^2)$

All four sorted correctly but took different amounts of work. The $O(n \log n)$ ones stayed close while shaker sort fell behind at $n = 8$. Implementing them made the differences a lot more concrete than just reading about them.

The main takeaway is you need to understand an algorithm before you try to code it. Wrong steps produce bad output and tracing it back through recursive calls is tedious. Next time id walk through an example by hand first.

- Course materials and previous projects
- YouTube videos on merge sort quick sort heap sort and shaker sort
- cpreference.com for C++ standard library documentation